03 · HANDS-ON

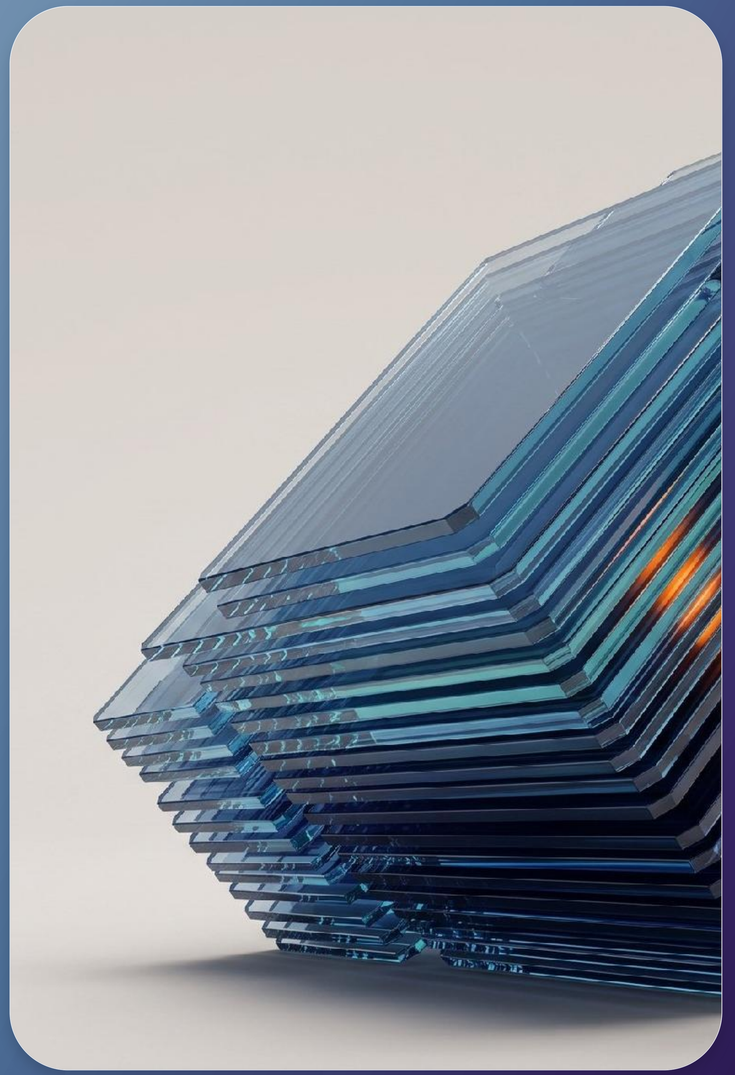
9:40 - 9:50 · 10 minutes

Lab 2 — Storage System Configuration

Misconfigure a filesystem — then fix it, live

One lever, end to end: striping, with a before/after view. The full workbook ships in the tutorial download tree (HandsOnLab2/).

TEXT EDITOR ONLY | NO CLUSTER | PRE-RECORDED RESULTS



ddn

LAB 2 · THE METHOD

How the lab works: from state to outcome

01

Current state

Read the system before you touch it.

02

Prompt

The exact command or question. Read it — nothing here runs.

03

Output

The pre-recorded result. Open the file, find the line, note the number.

04

Outcome

What it proves — as a multiple of the baseline.

SIX STEPS · CLICK TO JUMP

1 · SPOT THE PROBLEM	2 · SEE THE FIX	3 · MEASURE IT	4 · ADVANCED LAYOUTS	5 · READ THE SYSTEM
LIVE	LIVE	LIVE	LIVE	TAKE-HOME

HANDSONLAB2/

0-reference

settings-tables · systems-matrix · decision-table

1-base-config

annotated.conf · lfs-df.out · lctl-dl.out · cluster-map

2-striping

commands.sh · before.out · after.out · results.csv

3-advanced-striping

pfl · dom · pools · results-summary

4-metadata

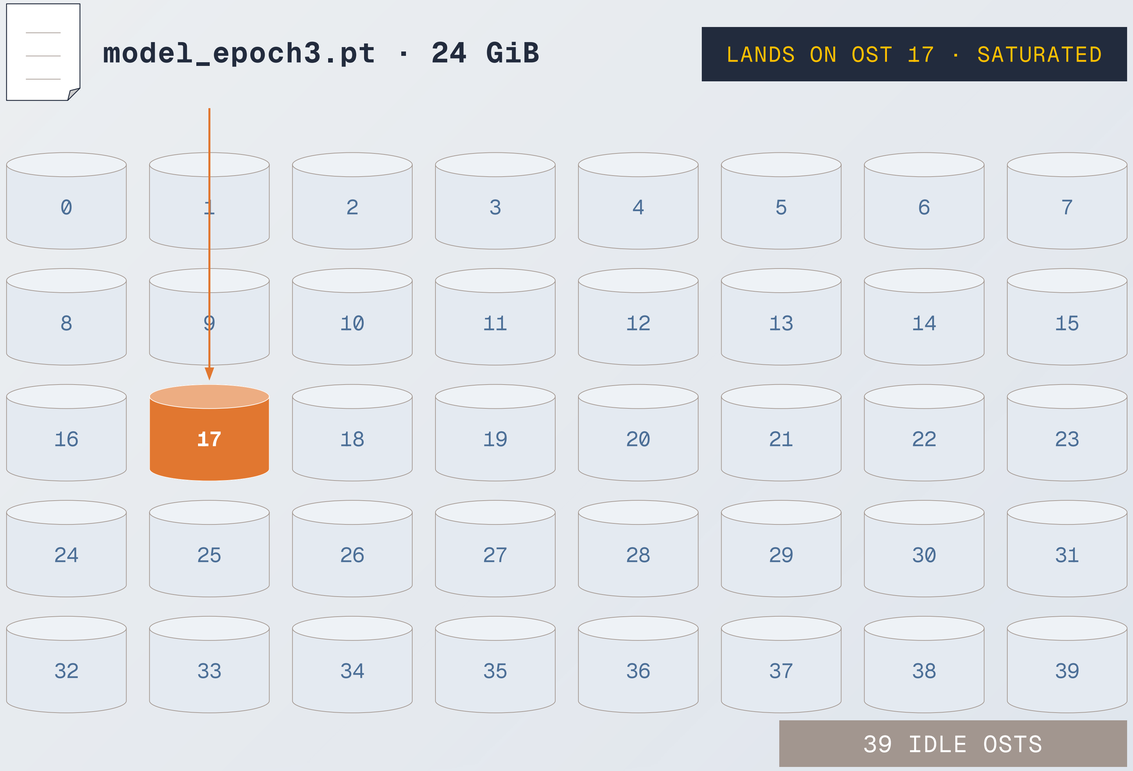
dne · nodemap · project-quota · changelog

LAB MAP ↗

Step 1 · One file, one target — the default

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME

AI0 · 40 OSTS · ONE NAMESPACE



1 of 40

OSTs doing any work

“Your 40-OST filesystem just performed like a single disk shelf.”

`stripe_count = 1` means the whole file lives on one server’s worth of bandwidth — no matter how many targets you own.

OST Object Storage Target **GiB** gibibyte (2^30 bytes) **.pt** PyTorch checkpoint

LAB MAP ↗

Step 1 · Ask: how is this file laid out?

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME

~/HandsOnLab2/2-striping · before.out

```
$ lfs getstripe /scratch/ckpt/model_epoch3.pt

# how is this file laid out across the OSTs?


$ ior -w -t 1m -b 24g -o /scratch/ckpt/model_epoch3.pt

# write the 24 GiB checkpoint — how fast, vs. baseline?
```

LFS GETSTRIPE

How is this file laid out across the OSTs?

IOR -W

How fast does the write go — relative to the baseline?

OPEN THIS FILE

2-striping/before.out

pre-recorded — no cluster, no install, just a text editor.

COMMNADS

lfs = GIT Large File Storage Command

Ior = Interleaved-Or-Random I/O Benchmark

LAB MAP ↗

Step 1 · Read it: stripe_count = 1

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME

before.out

```
$ lfs getstripe /scratch/ckpt/model_epoch3.pt
  lmm_stripe_count:  1                                   A
  lmm_stripe_size:   1048576
  lmm_pattern:       raid0
  lmm_stripe_offset: 17
$ ior -w -t 1m -b 24g -o /scratch/ckpt/model_epoch3.pt
Operation   Max(Rel)  Min(Rel)  Mean(Rel)  StdDev  OSTs touched
write          1.00      1.00       1.00    0.01             1   B
# Only OST 17 did any work. 39 OSTs sat idle.
```

1.0×

baseline — everything else is measured against this

A **lmm_stripe_count: 1**

the whole file on ONE OST

B **OSTs touched: 1**

one server's worth of bandwidth

LAB MAP ↗

Step 1 outcome: you found the trap

1

OST TOUCHED

One file → one OST → one server's worth of bandwidth, no matter how many targets the file system has.

39

OSTS IDLE

A 40-OST system performing like a single disk shelf. This is the Lustre default, and it is the trap.

1.0×

THE BASELINE

Every later result in this lab is a multiple of this number. Relative deltas are enough to prove the point.

NEXT: SEE THE FIX ↗

LAB MAP ↗

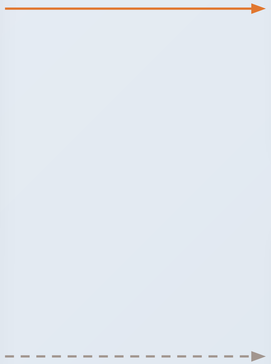
Step 2 · The fix is policy on a directory

/scratch/ckpt/

DIRECTORY POLICY

`STRIPE_COUNT = 1`

The lever lives on the directory — not on the file.





NEW FILES

Inherit the directory policy

Anything created from now on picks up the new stripe layout automatically.



EXISTING FILES · MODEL_EPOCH3.PT

Keep their old layout

Until they are rewritten — copy out and back, or migrate in place.

```
$ lfs migrate -c 8 /scratch/ckpt/model_epoch3.pt      # migrate in place instead of copying out and back
```

LAB MAP ↗

Step 2 · Prompt: widen the stripe, then go all the way

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME

2-striping/commands.sh

```
# widen the stripe on the directory
$ lfs setstripe -c 8 /scratch/ckpt/
# spread new files across 8 OSTs
# ... application re-writes the checkpoint ...
# go all the way: every OST
$ lfs setstripe -c -1 /scratch/ckpt/
# -1 = all 40 OSTs · re-write again
$ lfs getstripe /scratch/ckpt/model_epoch3.pt
# confirm the layout actually changed
```

-c -1

means “stripe across every OST.” The directory carries the policy; new files inherit it.

JUMP TO THE DASHBOARD

RUN -C 8 ↗

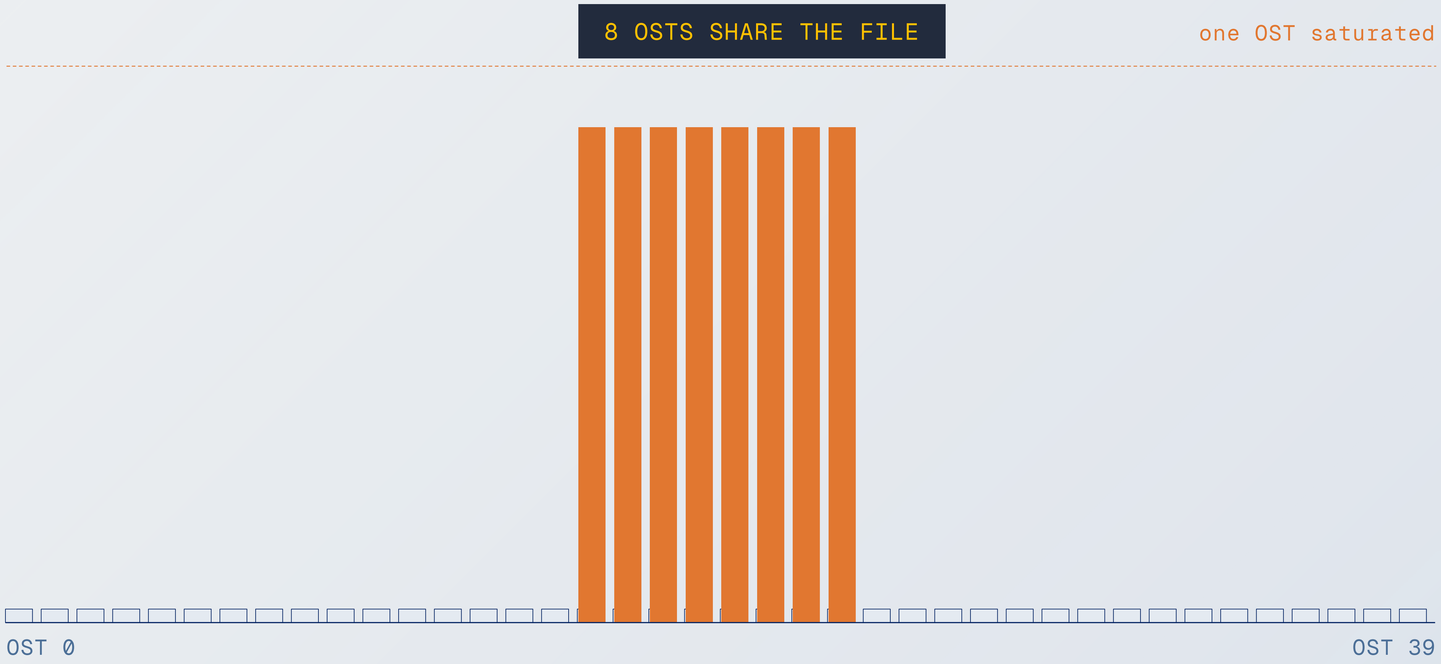
RUN -C -1 ↗

LAB MAP ↗

Step 2 · Stripe it — watch the dashboard flip

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME

PER-OST LOAD · 40 OSTS · RELATIVE UNITS



Illustrative dashboard in relative units — same workload, same hardware; only the stripe layout changed.

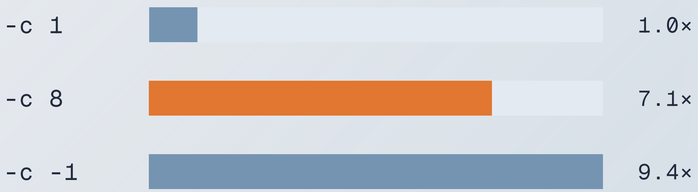
-C 1 | -C 8 | -C -1

7.1×

write throughput vs. baseline

OSTS TOUCHED: 8

WRITE THROUGHPUT (RELATIVE)

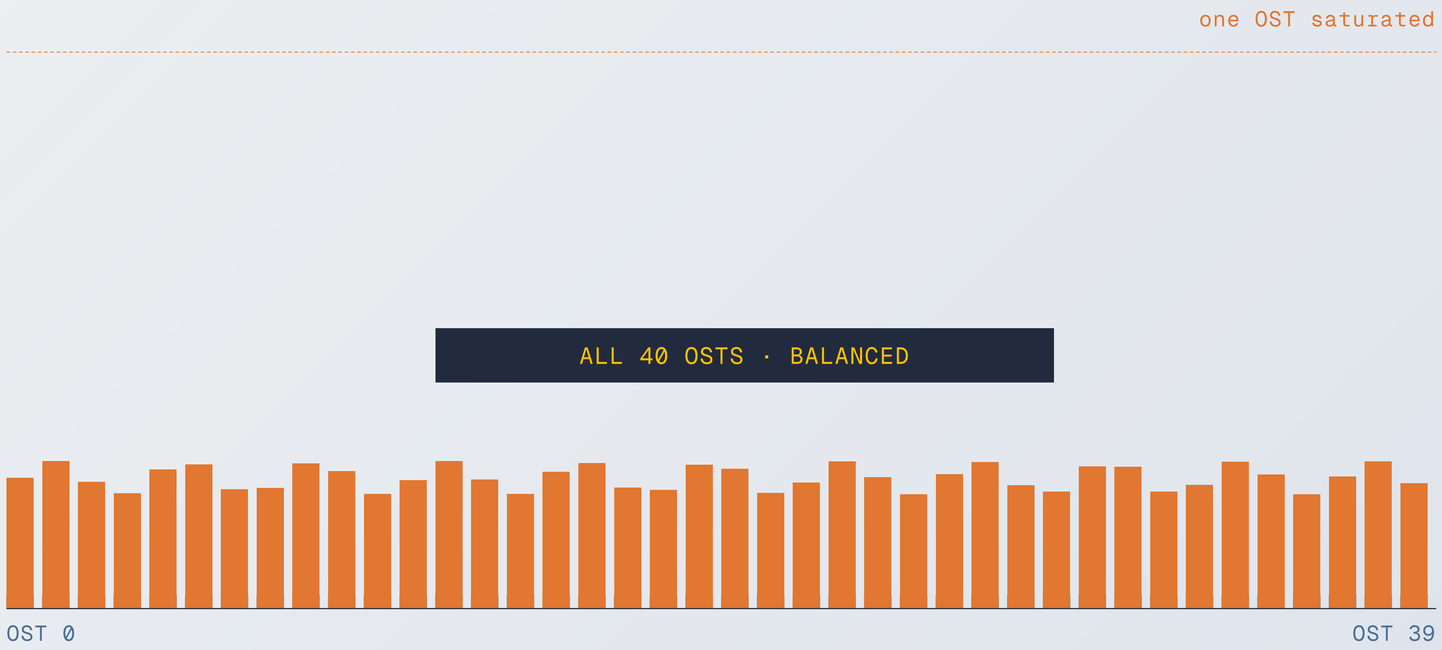


~linear with stripe width, then tapers

LAB MAP ↗

Step 2 · Stripe it — watch the dashboard flip

PER-OST LOAD · 40 OSTS · RELATIVE UNITS



Illustrative dashboard in relative units — same workload, same hardware; only the stripe layout changed.

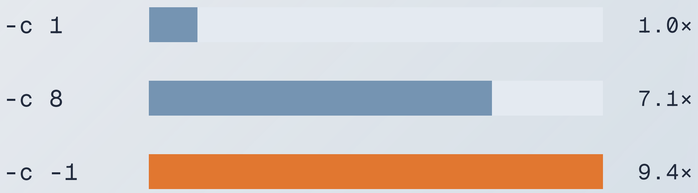
-C 1 | -C 8 | -C -1

9.4×

write throughput vs. baseline

OSTS TOUCHED: 40

WRITE THROUGHPUT (RELATIVE)

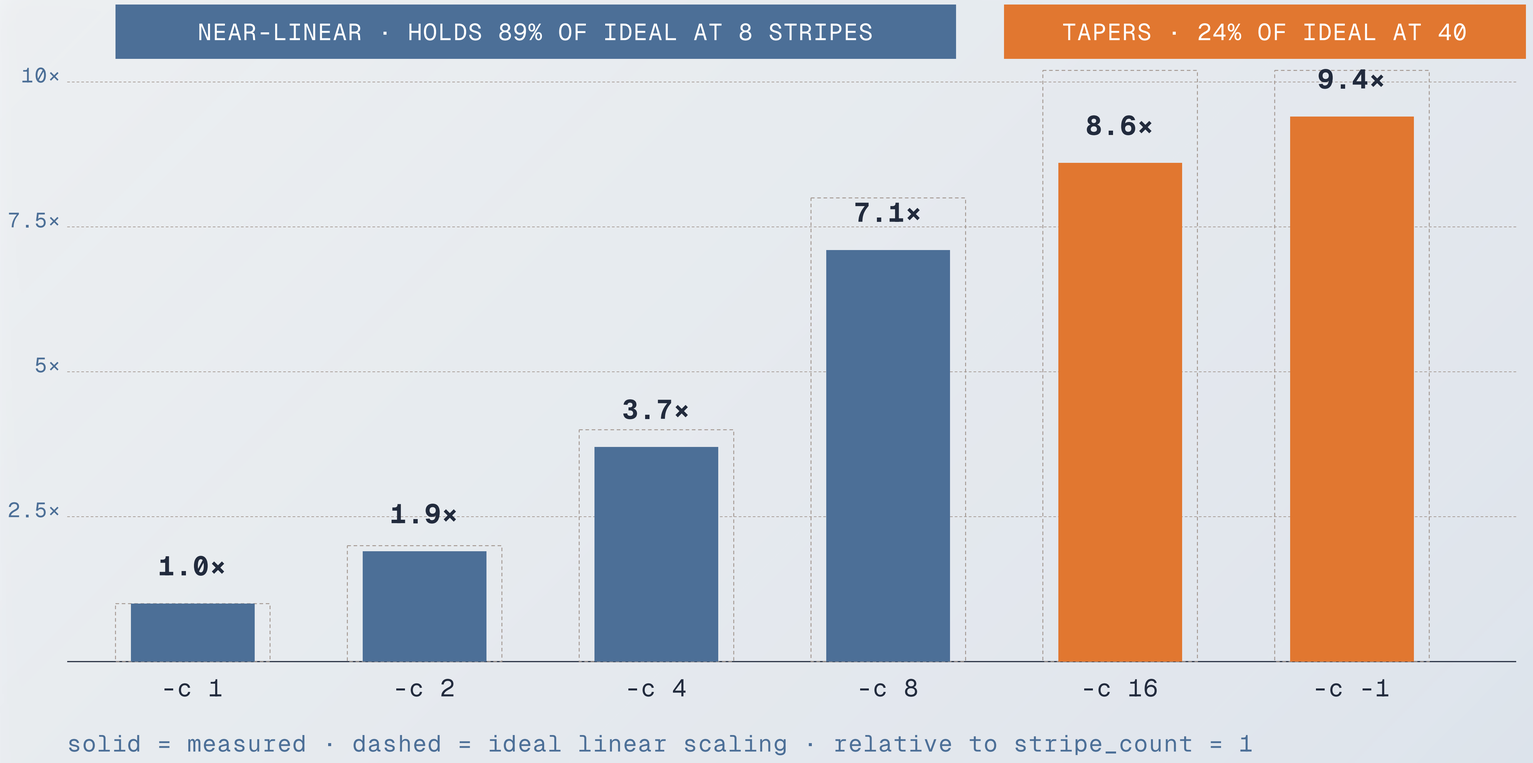


diminishing returns: network + client cap

LAB MAP ↗

Step 3 · Read the curve: near-linear, then a wall

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME



89%

of ideal at 8 stripes
(7.1 ÷ 8)

24%

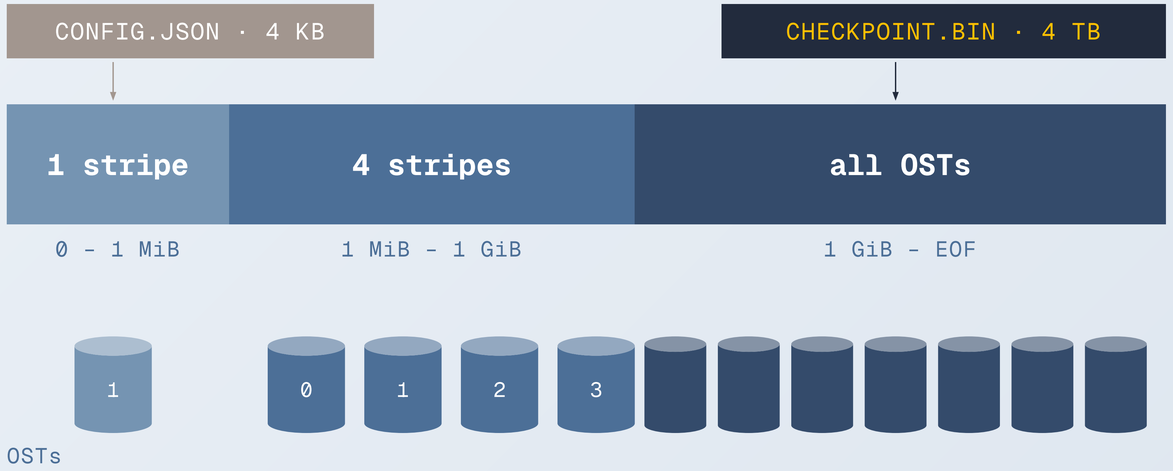
of ideal at 40 stripes
(9.4 ÷ 40)

Bandwidth scales with stripe width until another resource becomes the wall.

LAB MAP ↗

Step 4 · Files aren’t all one size: PFL

Wide striping is right for a 4 TB checkpoint — and wrong for a 4 KB config file.



One policy, any file size. Small files and file heads stay cheap; huge files fan out fully. Users stop guessing.

pfl-example.sh

```
$ lfs setstripe \
    -E 1M  -c 1  \
    -E 1G  -c 4  \
    -E -1  -c -1 \
    /scratch/mixed/
# layout follows file size
```

WHY

The stripe pattern changes as the file grows. One policy fits a 4 KB config and a 4 TB checkpoint.

Step 4 · Output: composite layouts, DoM and pools

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME

GETSTRIPE · PFL

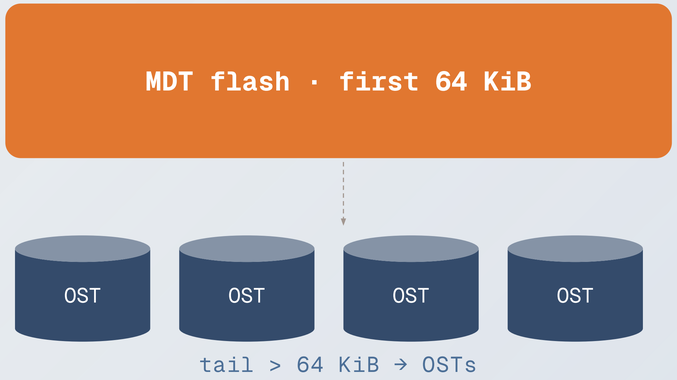
composite file

```
lcm_layout_gen:    3
lcm_entry_count:   3
e_end: 1048576
  stripe_count: 1
e_end: 1073741824
  stripe_count: 4
e_end: EOF
  stripe_count: -1
```

getstripe now prints one block per component.

DATA-ON-MDT (DOM)

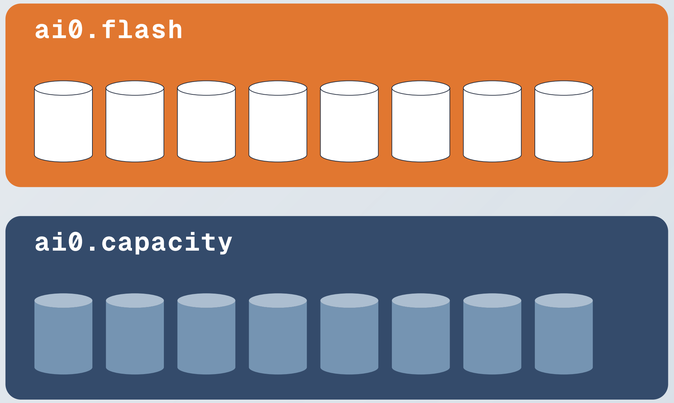
```
lfs setstripe -E 64K -L mdt
              -E -1 -c 4 /scratch/smallfiles/
```



Small files never pay an OST round-trip.
Trade: MDT flash is scarce — use where small-file latency dominates.

OST POOLS

```
lctl pool_new ai0.flash
lctl pool_add ai0.flash ai0-OST[0-7]
lfs setstripe -p ai0.flash -c 8 /scratch/hot/
```



Pools + PFL: users never choose width or media by hand.

LAB MAP ↗

Step 4 outcome: what each layout buys you

01 CURRENT STATE | 02 PROMPT | 03 OUTPUT | 04 OUTCOME

LAYOUT	SMALL-FILE OPEN	LARGE SEQUENTIAL WRITE	BLAST RADIUS
stripe_count = 1 (default)	1.0×	1.0×	● ○ ○ smallest
stripe_count = 8 (flat wide)	0.9×	7.1×	● ● ○ medium
stripe_count = -1 (all OSTs)	0.7×	9.4×	● ● ● largest
PFL (1 / 4 / all by size)	1.0×	9.0×	● ● ○ grows with size
PFL + DoM (64K on MDT)	3.2×*	9.0×	● ● ● small tail only

* Small-file gain comes from skipping OST round-trips for the tiny tail (DoM). ⚑ *Availability math is deployment-specific — treat blast radius as a direction, not a number.*

“Striping buys bandwidth with your availability math — know what you’re spending.”

LAB MAP ↗

Six habits to remember from this lab

1

Know your workload first

which KPI do you live or die on?

2

Stripe wide for big sequential, narrow for small

the default of 1 is the trap

3

PFL + pools so policy is automatic

layout follows file size, not user guesswork

4

Size the metadata plane like you mean it

mass-open jobs and governance both live there

5

Tune the network too

LNet multi-rail, RDMA/RoCE, GDS — the fabric caps your stripe

6

Measure before and after every change

relative deltas are enough to prove the point

Next: emerging storage tech (S3oRDMA, cuObject), GPU-accelerated storage, then KV cache.

PFL Progressive File Layout **LNet** Lustre Networking **S3oRDMA** S3 over RDMA **RDMA** Remote Direct Memory Access **RoCE** RDMA over Converged Ethernet **GDS** GPUDirect Storage **KPI** Key Performance Indicator **KV** Key-Value